МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»
Московский приборостроительный техникум

ОТЧЕТ

по практической работе №5

Тема: Аналоговые датчики, analogRead(), map()

Специальность 09.02.07 «Информационные системы и программирование»
код и наименование специальности

Студент: **Девлет Максим Керимович**
(фамилия, имя, отчество)

Группа: П-1-24

Руководитель по дисциплине:
Белова Дарья Романовна
(фамилия, имя, отчество)

«09» ноября 2026 года

ПРАКТИЧЕСКАЯ РАБОТА №5

Тема: Аналоговые датчики, analogRead(), map()

5.1. Цель работы: изучить принципы функционирования аналоговых датчиков и основы аналого-цифрового преобразования (АЦП) на базе 10-разрядного АЦП последовательного приближения (SAR ADC) микроконтроллера ATmega328P платформы Arduino. Освоить практическое применение встроенных функций считывания аналогового напряжения analogRead(), ограничения диапазона constrain() и линейного масштабирования map(). Спроектировать и смоделировать аппаратно-программную систему автомобильного парктроника («датчик парковки») на основе фоторезистора, пьезоизлучателя, светодиода и тактовой кнопки с неблокирующей логикой работы (millis()), автоматической 3-секундной калибровкой базового уровня освещенности, зависимостью частоты импульсов от приближения руки, режимом непрерывной аварийной сигнализации («СТОП!») при сближении менее 10% и асинхронным отключением звукового сопровождения через аппаратное прерывание (attachInterrupt()).

5.2. Схема подключения в эмуляторе

Интерактивный проект схемы доступен в среде Cirkit Designer по ссылке: https://app.cirkitdesigner.com/project/41090fec-b92a-40f8-b21c-a15b956f2572

Схема устройства смоделирована на базе микроконтроллера Arduino Uno в среде симуляции Cirkit Designer / Tinkercad. Подключение внешних компонентов организовано следующим образом:
• Фоторезистор (LDR): включен по схеме резистивного делителя напряжения с постоянным резистором 10 кОм (вывод к +5V, средняя точка к аналоговому входу A0, резистор 10 кОм к GND). При приближении руки освещенность падает, сопротивление растет, напряжение на входе A0 снижается;
• Тактовая кнопка: подключена между цифровым выводом 2 (BUTTON_PIN) и шиной GND со встроенной подтяжкой INPUT_PULLUP (аппаратное прерывание INT0, фиксация нажатия по спадающему фронту FALLING);
• Пьезоизлучатель (Buzzer): подключен к цифровому выводу 8 (BUZZER_PIN) и шине GND. Генерация звукового сигнала частотой 2000 Гц осуществляется функцией tone(), выключение — noTone();
• Светодиод (LED): анод через токоограничивающий резистор 220 Ом подключен к цифровому выводу 9 (LED_PIN), катод соединен с шиной GND;
• Скорость передачи телеметрии в COM-порт (UART): 9600 бод.

Таблица 1. Распределение выводов и параметров схемы аналогового парктроника

Пин Arduino	Подключенный компонент	Назначение и логика работы	Режим (Mode)
A0	Фоторезистор (LDR) + резистор 10 кОм	Делитель напряжения; измерение освещенности (0–1023)	INPUT (Analog ADC)
D2 (INT0)	Тактовая кнопка (замыкание на GND)	Переключение звука (Mute); аппаратное прерывание по фронту FALLING	INPUT_PULLUP
D8	Пьезоизлучатель (Buzzer)	Звуковая сигнализация (импульсы 2000 Гц / непрерывный тон при сближении)	OUTPUT
D9	Светодиод (LED) через резистор 220 Ом	Световая визуализация импульсов приближения и непрерывная тревога	OUTPUT
GND / +5V	Общие шины питания	Питание делителя фоторезистора и замыкание цепей периферии	Power
UART	Serial Monitor	Вывод телеметрии (каждые 300 мс) и предупреждения «СТОП!»	9600 бод

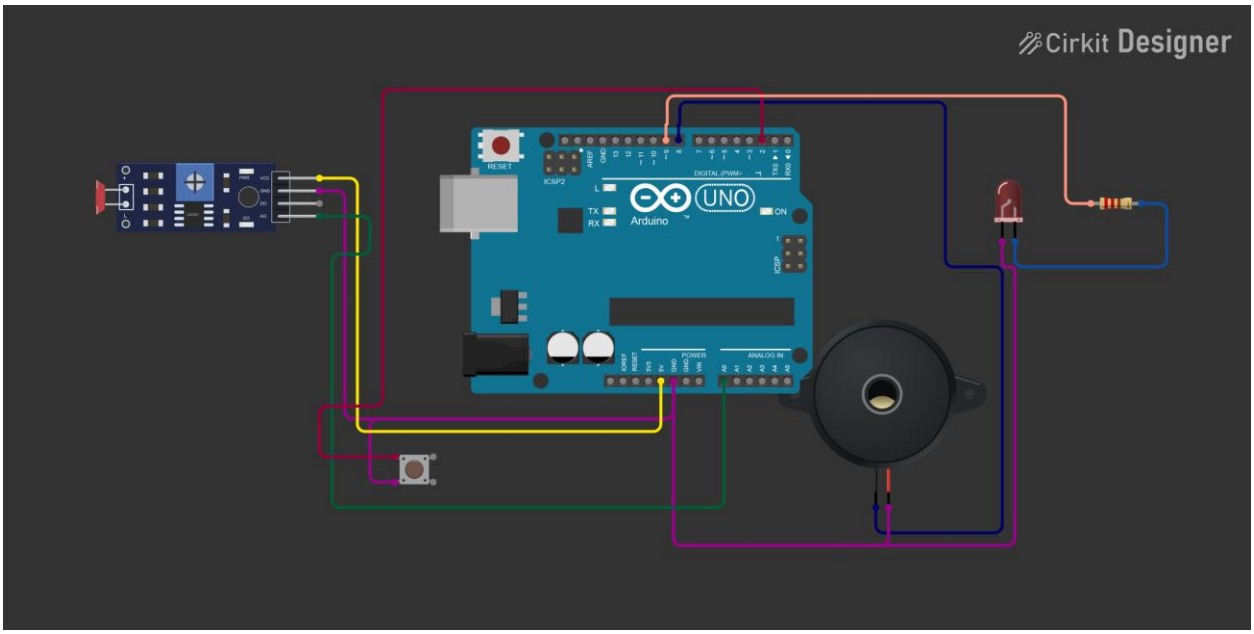


Рис. 1. Принципиальная схема подключения аналогового парктроника к Arduino Uno в эмуляторе

5.3. Исходный код программы

```
const uint8_t LDR_PIN = A0;
const uint8_t BUTTON_PIN = 2;
const uint8_t BUZZER_PIN = 8;
const uint8_t LED_PIN = 9;

const uint32_t CALIBRATION_MS = 3000UL;
const uint32_t DEBOUNCE_MS = 180UL;
const uint32_t SERIAL_INTERVAL_MS = 300UL;
const uint32_t BEEP_DURATION_MS = 60UL;
const uint16_t TONE_FREQ = 2000U;

uint32_t calibrateStartTime = 0;
uint32_t lastSignalTime = 0;
uint32_t lastSerialTime = 0;

volatile uint32_t lastButtonTime = 0;
volatile bool soundEnabled = true;

int calibratedLight = 0;
int minDark = 0;
uint32_t calibrationSum = 0;
uint16_t calibrationSamples = 0;

bool isCalibrated = false;
bool signalActive = false;

void toggleSound() {
  const uint32_t now = millis();
  if (now - lastButtonTime >= DEBOUNCE_MS) {
    soundEnabled = !soundEnabled;
    lastButtonTime = now;
  }
}

void setup() {
  pinMode(LDR_PIN, INPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(BUZZER_PIN, OUTPUT);
  pinMode(LED_PIN, OUTPUT);

  digitalWrite(LED_PIN, LOW);
  noTone(BUZZER_PIN);

  Serial.begin(9600);
  calibrateStartTime = millis();
  attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), toggleSound, FALLING);

  Serial.println(F("Калибровка освещения: 3 секунды..."));
}

void loop() {
  const uint32_t now = millis();

  if (!isCalibrated) {
    if (calibrationSamples < 60000U) {
      calibrationSum += analogRead(LDR_PIN);
      ++calibrationSamples;
    }
```

```
  if (now - calibrateStartTime >= CALIBRATION_MS) {
    if (calibrationSamples == 0) return;

    calibratedLight = calibrationSum / calibrationSamples;
    minDark = calibratedLight - 300;
    if (minDark < 0) minDark = 0;
    if (calibratedLight <= minDark) calibratedLight = minDark + 1;

    isCalibrated = true;
    Serial.print(F("Калибровка завершена; базовый ADC = "));
    Serial.println(calibratedLight);
  }
  return;
}

const int val = analogRead(LDR_PIN);
const int constrainedVal = constrain(val, minDark, calibratedLight);
const int distance = map(constrainedVal, minDark, calibratedLight, 0, 100);
const bool alarm = (distance < 10);

if (now - lastSerialTime >= SERIAL_INTERVAL_MS) {
  lastSerialTime = now;
  if (alarm) {
    Serial.println(F("СТОП!"));
  } else {
    Serial.print(distance);
    Serial.println(F("%"));
  }
}

if (alarm) {
  digitalWrite(LED_PIN, HIGH);
  if (soundEnabled) {
    tone(BUZZER_PIN, TONE_FREQ);
  } else {
    noTone(BUZZER_PIN);
  }
  signalActive = true;
  lastSignalTime = now;
} else {
  const uint32_t pauseMs = (uint32_t)map(distance, 10, 100, 100, 1000);

  if (signalActive) {
    if (now - lastSignalTime >= BEEP_DURATION_MS) {
      signalActive = false;
      lastSignalTime = now;
      digitalWrite(LED_PIN, LOW);
      noTone(BUZZER_PIN);
    } else if (!soundEnabled) {
      noTone(BUZZER_PIN);
    }
  } else {
    if (now - lastSignalTime >= pauseMs) {
      signalActive = true;
      lastSignalTime = now;
      digitalWrite(LED_PIN, HIGH);
      if (soundEnabled) {
        tone(BUZZER_PIN, TONE_FREQ);
      }
```

```
      }
    }
  }
}
```

5.4. Краткое описание алгоритма

В функциях setup(), loop() и процедуре прерывания toggleSound() реализован неблокирующий масштабируемый алгоритм управления парковочной системой:
1. Инициализация и регистрация прерывания (setup): аналоговый вход A0 настраивается на ввод (INPUT), пин кнопки 2 конфигурируется со встроенной подтяжкой (INPUT_PULLUP), выводы зуммера (8) и светодиода (9) переводятся в режим вывода (OUTPUT). Функция attachInterrupt(digitalPinToInterrupt(BUTTON_PIN), toggleSound, FALLING) связывает внешнее прерывание INT0 с обработчиком кнопки. Устанавливается скорость UART 9600 бод и фиксируется метка времени старта калибровки.
2. Адаптивная автокалибровка (isCalibrated == false): в течение первых 3000 мс программа накапливает сумму отсчетов АЦП (calibrationSum) и число выборок (до 60000). По истечении таймера вычисляется среднее значение calibratedLight (базовый уровень освещенности помещения в состоянии «свободно»), а также адаптивный порог затемнения minDark = calibratedLight - 300 с защитой от выхода в отрицательную область.
3. Опрос датчика и расчет расстояния (loop): функция analogRead(LDR_PIN) считывает уровень сигнала. Через constrain(val, minDark, calibratedLight) исключаются случайные выбросы за пределы калибровки, а с помощью map(constrainedVal, minDark, calibratedLight, 0, 100) значение преобразуется в процентное «расстояние» от 0% («рука вплотную») до 100% («препятствия нет»). Вычисляется условие аварийной зоны alarm = (distance < 10).
4. Интервальная телеметрия UART: каждые 300 мс по системному таймеру (now - lastSerialTime >= SERIAL_INTERVAL_MS) в COM-порт выводится либо текущее расстояние в процентах, либо предупреждение «СТОП!», если активирован режим alarm.
5. Неблокирующая светозвуковая сигнализация: при distance < 10% светодиод горит постоянно, а зуммер выдает непрерывный тон 2000 Гц (если звук разрешен). При distance >= 10% длительность паузы между короткими импульсами (BEEP_DURATION_MS = 60 мс) динамически масштабируется через map(distance, 10, 100, 100, 1000) от 100 мс (близко) до 1000 мс (далеко). Переключение состояний выполняется через millis() без задержек delay().
6. Асинхронное переключение звука (toggleSound): по спадающему фронту

нажатия кнопки вызывается обработчик прерывания. Проверка now - lastButtonTime >= DEBOUNCE_MS (180 мс) подавляет дребезг контактов. Переменная soundEnabled инвертируется; спецификатор volatile гарантирует моментальное считывание актуального состояния в основном цикле, при этом светодиодная индикация продолжает функционировать независимо от звука.

5.5. Вывод: в ходе выполнения практической работы были всесторонне изучены принципы взаимодействия с аналоговыми датчиками и механизмы АЦП в микроконтроллерах ATmega328P на платформе Arduino. Спроектирована и промоделирована в среде Cirkit Designer схема автомобильного парктроника с фоторезистивным делителем на входе A0, кнопкой внешнего прерывания INT0 (пин 2), пьезоизлучателем (пин 8) и светодиодом (пин 9). Успешно применены функции analogRead(), constrain() и map() для пересчета аналогового сигнала в условную дистанцию от 0% до 100% и динамической регулировки интервалов звуковых импульсов от 100 до 1000 мс. Разработан полностью неблокирующий алгоритм на базе системного таймера millis(), включающий 3-секундную адаптивную автокалибровку под фоновый свет, непрерывную аварийную сигнализацию («СТОП!») при сближении менее 10%, периодический вывод телеметрии в последовательный COM-порт каждые 300 мс и асинхронное отключение звукового сопровождения через аппаратное прерывание с программным подавлением дребезга контактов.